Analog Speed Control System for DC Motor Applications

Derek Stoddard, EE
Swaroop Handral, EE

March 13, 2026

University of Maine

ECE 403 Final Report

Abstract

This report presents the design, simulation, and hardware implementation of an analog proportional-integral (PI) controller and driver for a DC motor. The driver controls power supplied to the motor based on signals from the controller, utilizing a low-side MOSFET and gate driver to regulate operation. The controller uses analog pulse-width modulation (PWM) to maintain a desired speed under varying load conditions, without the use of a microcontroller. The design was validated through simulation in MATLAB before being built on solderless breadboards and tested with a physical DC motor. The final prototype was built on protoboards and powered with an external power supply. The system met all design specifications, demonstrating effective analog speed regulation.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Analog Speed Control: Background, Motivations, and Operation

Electric motors are one of the most widely used devices in modern technology. DC motors in particular are found in a wide array of applications, from household appliances to manufacturing equipment. While simple on/off operation is sometimes adequate, the motor must often be required to spin at a specific and consistent speed regardless of how much work it is being asked to do. Take as an example a conveyor belt in a factory. The belt must move at the same rate whether it is carrying a single package or hundreds to keep production running smoothly. Without some sort of speed control on the motor, the belt would slow down as the load it was carrying got heavier, and speed up as it was reduced.

A motor controller solves this problem by continuously monitoring the motor's speed and making adjustments to the power delivered to it via closed-loop feedback control. The controller compares actual speed of the motor to the desired speed (the "setpoint") and calculates the difference, or "error." The controller then adjusts its output to correct for the error. A proportional-integral (PI) controller is a common type of feedback controller that responds to both the instantaneous error and the accumulation of error over time. The "proportional" portion of the controller provides an immediate correction based on how large the error is, while the "integral" portion accounts for any small, persistent error that the proportional term cannot itself correct. These two components allow the controller to quickly and accurately bring the motor to the setpoint and hold it in steady-state.

This project focuses on designing and building a purely analog proportional-integral (PI) controller to regulate the speed of a small DC motor. While most modern motor controllers depend on embedded controllers and algorithms, this project achieves speed control exclusively using analog components. The team chose this approach due to a shared interest in control theory, industrial automation, and analog design.

At a high level, the system continuously measures the motor's speed, compares it to the desired speed, and adjusts power delivery to correct any difference. More specifically, the system operates as a closed-loop feedback controller. A potentiometer is used to set the desired motor speed as a reference voltage. An incremental encoder attached to the motor shaft produces a square-wave pulse signal proportional to the actual speed of the motor. The frequency of this pulse is converted to a DC voltage by a frequency-to-voltage (F/V) converter. The difference between the setpoint and the voltage from the F/V converter produces an error voltage, which is fed into the PI controller. The controller outputs a control voltage that drives a PWM generator, implemented using a triangle wave generator. The resulting PWM signal switches a low-side MOSFET through a gate driver, modulating the power delivered to the motor. This process repeats continuously, adjusting the motor

speed until the error between the setpoint and the actual speed is minimized.

The design of the project was guided by design specifications agreed upon during the first capstone semester. Regarding power requirements, it was decided that the driver circuit would be able to supply a minimum of one amp to the motor. The system would have to use a PI controller to regulate speed at $\pm 10\%$ of the setpoint at steady state across a range of 50 to 150 RPM. The controller would also have to maintain speed despite a reasonable change in load within that range. Additionally, an encoder would be used to measure the speed of the motor shaft. MATLAB was used to model and verify the transfer function of the DC motor, ensuring that the mathematical representation of the motor's behavior accurately reflected its physical characteristics. After the build phase, the specifications of the project were verified using an oscilloscope to capture and measure the waveform of the controller output and compare it against the setpoint voltage, measuring steady-state error. This was done under no load conditions as well as under load conditions. The power requirements were verified using a bench-top power supply and monitoring the drawn current as load was applied to the motor. Load was applied to the motor manually by grabbing a small wheel affixed to the shaft.

Similar commercially available devices are typically digital, utilizing microcontrollers and digital signal processing. At a consumer level, many of these devices include user interfaces, mobile or desktop software for configuration, and preset operating modes. At an industrial level, commercial motor controllers often include programmable speed profiles, automatic control-loop tuning, and diagnostic capabilities. Industrial controllers are used to operate large motors in manufacturing environments. Although there are more points of failure, these products are more practical for the majority of real-world applications due to their ease of use and flexibility. The PI controller developed in this project is purely analog and relies on manual, "guess and check" style tuning, exchanging flexibility and ease of use for affordability. While real-world applications are limited, the analog controller was built using cheap, widely available components and could be a valuable device for small hobby DC motor applications with further improvements and additions.

1.2 Public Health, Safety, and Welfare

Feedback control systems provide motors with stable operation and predictable behavior under changing conditions, giving operators a safer environment to work in—sudden stalling or overspeeding can increase the risk of harm to people as well as damage to equipment. The regulation of speed through feedback helps to reduce these risks and increases safety and reliability. Feedback control also provides a greater degree of safety on the roads in the form of cruise control. Predictable drivers are safe drivers and by allowing for a metered and consistent speed, cruise control allows drivers to be predictable.

Since this project was developed as an educational prototype, its intended use is limited to supervised testing with a small DC motor. It is not intended for use outside of this scope. Therefore, safety consideration focused on reducing exposure to dangerous voltages for the user and protecting against short circuits and overcurrent to protect the device. The design accomplished this by using

an enclosed AC-DC power supply, secure terminal connections, proper grounding, and mounting on a non-conductive surface.

1.3 Global, Cultural, Social, Environmental, and Economic Factors

The modern design of motor control systems centers around microcontrollers, software, and integrated diagnostic features. While this approach has effectively made the systems "off the shelf" solutions, it has also created a requirement for specialized software that is often behind a licensing paywall. It has also abstracted away how the circuits work on a fundamental level. This project is a foundational version of modern motor controllers—it strips away the abstraction by achieving the same basic objectives using discrete analog circuitry. Transparency of design lowers the barrier to repairability and accessibility by giving direct access to the circuit without any specialized software or licensing. The fully analog design also provides a tangible benefit for students by creating a direct relationship between theory and hardware, deepening the understanding of control principles.

Motors are ubiquitous in industry, consuming a huge share of electricity and in turn contributing to operating costs. Controllers regulate power delivery, which increases both efficiency and consistency. Given the scale at which motors are used worldwide, even marginal efficiency gains become meaningful. These gains reduce energy consumption, lowering demand for power generation and natural resources. At the same time, they lower costs to the producer, in turn making goods produced at scale more affordable for the consumer. At the project level, the design uses inexpensive and broadly available components. This device is easily repairable with standard components and does not require specialized software Environmentally, the design reduces waste because repairs can be made on a component level rather than replacing an entire device.

1.4 Ethical Use of the Device

The device designed and produced for this project is a bench prototype with a clearly defined scope—education and demonstration. Since the scope is limited, the device lacks safety protections provided by commercial controllers, such as fault detection and physical barriers between components and the user. For this reason, it is imperative that the controller only be used in a controlled test bench environment to demonstrate control theory principles.

2 Breakdown & Theory

2.1 System Overview & Signal Flow

The system is a closed-loop feedback controller for DC motor speed regulation. An adjustable reference voltage is compared with a feedback voltage to create an error signal. The error signal then passes through the PI controller and is converted to a switching signal that drives a DC motor via pulse-width modulation. The actual speed of the motor is measured by converting the frequency of encoder pulses to a DC voltage, creating the feedback signal. In the following sections, each stage of the block diagram will be discussed.

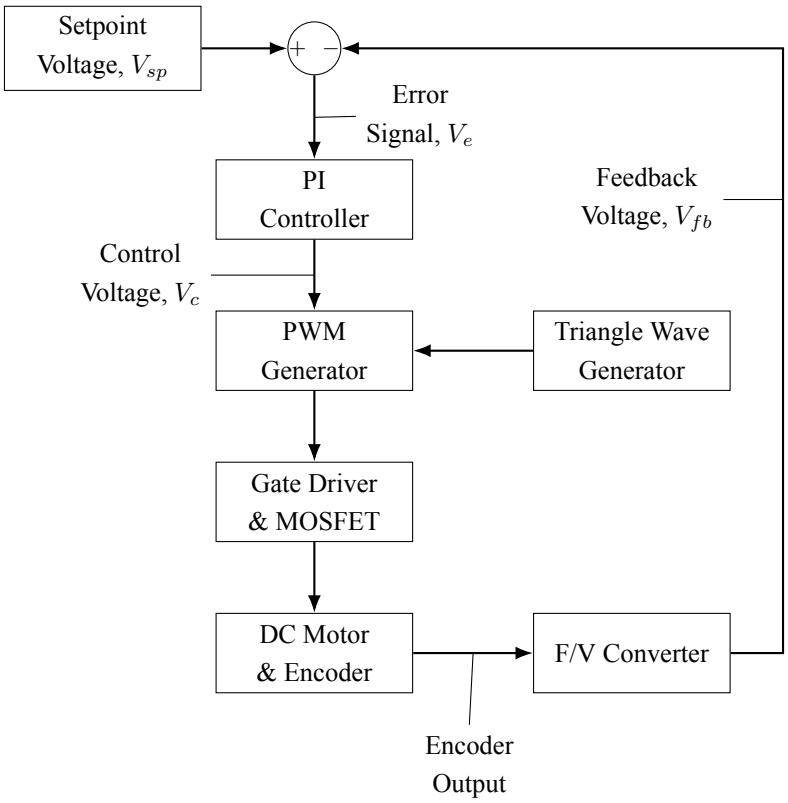

Figure 1: Block diagram of the closed-loop analog speed control system.

As shown in Figure 1, the closed-loop system starts with the Setpoint Voltage block. The setpoint voltage flows into the Summing Junction, where it is compared to the feedback voltage, producing the error signal. The error signal flows into the PI Controller block, which processes the error to create the control voltage. The control voltage enters the PWM Generator block along with the triangle wave from the Triangle Wave Generator block. The two inputs are compared, creating a square wave which flows into the Gate Driver & MOSFET block, driving the gate driver that

switches the MOSFET. The switching MOSFET modulates the power to the motor, controlling the speed. The rotation of the motor shaft is measured by the encoder, which outputs a square wave. The frequency of the encoder output is received by the frequency-to-voltage (F/V) Converter block, where it is measured and converted into a DC voltage, creating the feedback signal which flows back to the Summing Junction and closes the loop.

The setpoint voltage sets the desired speed of the motor by providing a reference voltage of 0-12 V_{DC} to the control system. In this project, a potentiometer is used as a voltage divider across the +12 V_{DC} regulated supply, producing an adjustable setpoint. Changing the potentiometer position changes the magnitude of the reference voltage, which in turn changes the motor speed setpoint. The setpoint voltage serves as the input to the control system and is continuously compared with the feedback signal that represents the actual motor speed.

The summing junction receives the setpoint voltage and the feedback voltage from the F/V converter. The two voltages are compared to produce an error signal:

$$V_e = V_{sp} - V_{fb} \tag{1}$$

The error signal represents the magnitude and direction of the difference between the setpoint and actual speed. When the motor is running below the setpoint speed, the feedback voltage is lower than the reference voltage, creating a positive error signal. When the motor is running above the setpoint, the feedback voltage is higher than the reference voltage, creating a negative error signal. If the motor is running at the desired speed, the feedback voltage matches the setpoint and the error approaches zero. The summing junction outputs the error signal to the controller to determine how the motor drive should be adjusted.

2.2 PI Controller

The PI controller receives the error signal from the summing junction and uses it to determine how strongly the motor should be driven. The controller consists of a proportional stage and an integral stage in parallel, and produces a control voltage:

$$V_c(t) = k_p V_e(t) + k_i \int V_e(t)dt.$$

In the Laplace domain:

$$V_c(s) = V_e(s)(k_p + \frac{k_i}{s}).$$

The proportional stage produces an output directly proportional to the instantaneous error, allowing the controller to respond immediately to a deviation from setpoint. As the name suggests, this response is proportional to the size of the difference—a larger variation produces a stronger correction. The integral stage accumulates error over time and increases the controller output to eliminate smaller steady-state error that the proportional stage may have left behind. By combining the re-

sponses of each stage, the controller is able to produce fast corrective action as well as accurate steady-state speed regulation. The controller outputs the control voltage to the PWM modulator.

2.3 Motor Driving and Feedback Paths

2.3.1 Triangle Wave Generator

The triangle wave generator is an oscillator that produces a periodic triangle waveform at a fixed frequency. The waveform serves as the carrier signal for the PWM stage. The oscillation frequency of the triangle wave sets the switching frequency of the PWM signal, ultimately setting the switching frequency for the motor driver stage.

2.3.2 PWM Generator

The PWM generator receives the control voltage from the PI controller and the triangle wave from the triangle wave generator. The control voltage is compared against the triangle wave, resulting in a square wave whose frequency is constant, but the duty cycle varies with the control voltage amplitude. A higher control voltage results in a higher duty cycle, which provides more power to the motor. This square wave is sent to the gate driver and ultimately regulates the power delivered to the motor.

2.3.3 Gate Driver & MOSFET

The gate driver and MOSFET stage receive the PWM control signal and use it to regulate the power delivered to the DC motor. The PWM waveform is applied first to the gate driver. The gate driver serves as a buffer between the PWM generation stage and the MOSFET, isolating the PWM stage from the MOSFET gate load. The gate driver then reproduces the PWM waveform as a gate-drive signal for the MOSFET. The MOSFET is a low-side switch in the motor current path. When turned on, the MOSFET creates a path to ground, allowing current to flow from the +12V_{DC} supply to the motor. When the MOSFET is switched off, this flow is interrupted. The signal from the gate driver repeatedly switches the MOSFET on and off based on the duty cycle of the PWM waveform. The output of this stage is a switched motor-drive signal.

2.3.4 DC Motor & Encoder

The DC motor is the ”plant” in the control system whose speed is being regulated by the controller. It receives the driving signal from the MOSFET, rotating its shaft based on the average ”on” time of the waveform. The encoder is the speed-sensing element in the system, and is attached to the motor. The encoder produces a square wave, the frequency of which is proportional to the rotational speed of the motor shaft. This square wave from the encoder is output to the F/V converter where its frequency will be converted to a DC voltage used for feedback.

3 Design Details

A closed-loop speed control system was required to use a purely analog PI controller to maintain motor speed within $\pm 10\%$ of a setpoint at steady state from 50 to 150 RPM. The system was also required to maintain motor speed despite a reasonable change in load within the same range of speed. Additionally, the driver had to supply a minimum of 1 amp to the motor. Finally, the system was required to use an encoder to measure the speed of the motor shaft.

The following sections detail and discuss the design decisions made to meet these requirements.

3.1 Operational Amplifier Selection

The team knew that the design of the project would center around operational amplifiers (op-amps). The op-amps would be responsible for summing voltages to create the error signal, creating the PI stage, and buffering signals. The op-amp would need to be compatible with dual supply rails, have low offsets, and have enough bandwidth for a DC motor control loop. With the design requirements in mind, the team chose the Texas Instruments OPA2197 precision op-amp. The OPA2197 supports $\pm 2.25\,\text{V}$ to $\pm 18\,\text{V}$ supplies which would give the error and control signals headroom to operate [1]. Additionally, the OPA2197 has a typical input offset voltage of $\pm 25\,\mu\text{V}$, limiting the amplifiers from producing their own error. Minimizing all sources of error in a control system is important for accurate operation. Finally, the OPA2197 has a gain-bandwidth of $10\,\text{MHz}$, which is more than enough for a low frequency motor loop as it would not limit the performance of the loop.

3.2 PI Controller Implementation

The design contract stipulated that the system would implement a PI controller to regulate motor speed within the specified operating range. The controller would need to accept an error signal from the summing junction and output a control voltage for the PWM stage. The analog circuit used to implement the PI controller is shown in Figure 2.

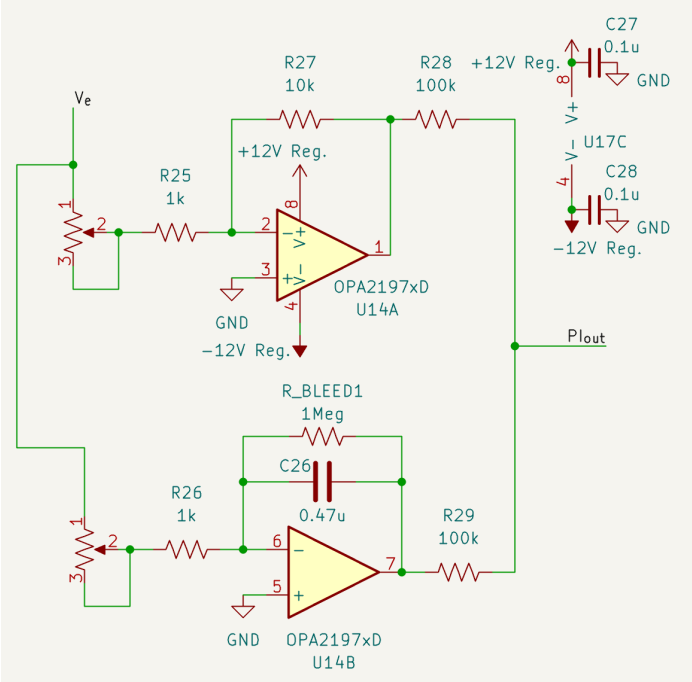

Figure 2: Analog PI controller implementation with OPA2197.

The team chose a parallel PI configuration, with separate proportional and integral branches driven by the same error signal $V_e(s)$. The outputs of the branches were summed to generate the control voltage. The team chose this design primarily because it would be easier to troubleshoot and tune the stages independently. The standard PI controller form can be viewed as,

$$G_c(s) = k_p + \frac{k_i}{s} \tag{2}$$

where k_p is the gain of the proportional stage and k_i is the gain of the integral stage. For the proportional stage, the gain is set by the ratio of the feedback resistance to the input resistance. As seen in Figure 2, the potentiometer combined with $R25$ created the input resistance for the P stage, while resistor $R27$ served as the feedback resistor.

$$R_{in,P} = R_{fixed,P} + R_{pot,P}, \tag{3}$$

so the resistance could not reach zero during tuning. Assuming ideal op-amp conditions, the proportional stage transfer function is

$$k_p = -\frac{R_{fb,P}}{R_{fixed,P} + R_{pot,P}}. \tag{4}$$

This shows that increasing or decreasing $R_{pot,P}$ directly changes the magnitude of the proportional gain, allowing for manual tuning.

The integral stage was implemented as an inverting integrator. As seen in Figure 2, the poten-

tiometer combined with fixed resistor $R26$ created the input resistance for the I stage and is written as

$$R_{in,I} = R_{fixed,I} + R_{pot,I}. \tag{5}$$

The feedback network consisted of a capacitor C_I, in parallel with a bleed resistor, R_{bleed}. The capacitor and input resistance set the integral gain. Since the capacitor's impedance is

$$Z_C(s) = \frac{1}{sC_I}, \tag{6}$$

its impedance approaches infinity at DC:

$$\lim_{s\to 0} Z_C(s) = \lim_{s\to 0} \frac{1}{sC_I} = \infty. \tag{7}$$

Adding the bleed resistor in parallel with the capacitor limits the low frequency gain of the integrator to

$$\frac{-R_{bleed}}{R_{in,I}}, \tag{8}$$

helping to prevent the integrator from saturating at the supply rails [2].

The ideal integrator gain can be expressed as

$$k_i = \frac{1}{(R_{fixed,I} + R_{pot,I})C_I}, \tag{9}$$

but due to the inclusion of a bleed resistor the transfer function of the stage becomes

$$H_I(s) = -\frac{R_{bleed}}{(R_{fixed,I} + R_{pot,I})(1 + sC_I R_{Bleed,I})}. \tag{10}$$

However, assuming that $sC_I R_{bleed} > 1$,

$$H_I(s) \approx -\frac{1}{sR_{in,I}C_I} \tag{11}$$

This shows that increasing or decreasing $R_{pot,I}$ directly adjusts the magnitude of the integral gain, allowing for manual tuning. The complete derivations of the PI transfer functions are included in Appendix A.

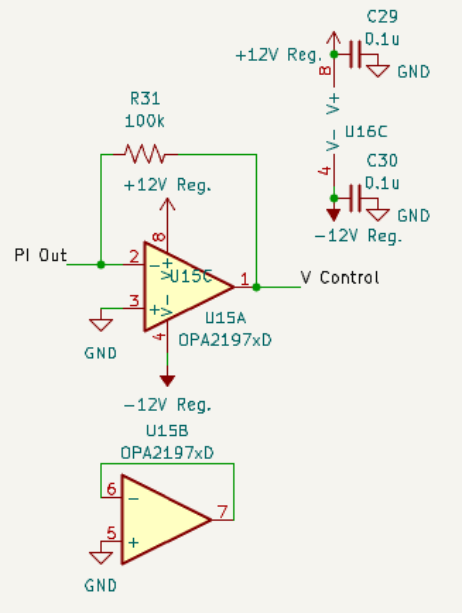

Figure 3: Implementation of summer circuit.

The controller output was formed by summing the outputs of the proportional and integral stages as shown in Figure 3.

3.3 Controller Gain Selection and Tuning

3.3.1 Design Objectives

The design objectives for the control loop were to keep a steady-state motor speed within $\pm 10\%$ of the setpoint from 50 to 150 RPM, and to also maintain this specification under reasonable load change. As a result, the primary design focus was minimizing steady-state error, while overshoot and settling time were treated as secondary considerations. Given the motor in the system had a maximum speed of approximately 300 RPM and the sensor scaling mapped 0–12 V to 0–300 RPM, the 50–150 RPM operating range corresponded to about 2–6 V of setpoint/feedback voltage range.

3.3.2 Loop Dynamics

The first consideration was to determine what components of the system would guide zero placement and gain choices. This started with the motor transfer function.

Motor Model

The motor is modeled in two parts—electrical and mechanical. Mechanically, the motor is modeled as a rigid rotating mass. Electrically, the motor is modeled as a voltage source in series with an inductor, resistor and second voltage source representing the counter-electromotive force (back EMF), as shown in Figure 4.

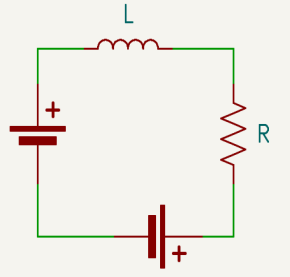

Figure 4: Electrical representation of a DC motor.

Using a combination of calculated and manufacturer provided parameters, the transfer function could be obtained using the standard armature-controlled DC motor model [3]:

$$G(s) = \frac{0.257}{(1.22 \cdot 10^{-6})s^2 + (1.04 \cdot 10^{-3})s + 0.07}. \tag{12}$$

From the transfer function, the motor poles were found to be 11.73 Hz and 123.95 Hz, with the 11.73 Hz pole being dominant. The full derivation of the motor transfer function and calculation of poles is included in Appendix B. In addition to the motor, the feedback path also contributed to the loop dynamics.

F/V Converter Pole

Since the encoder signal frequency was converted to a DC voltage by the F/V converter in the feedback path, the pole introduced by the converter's output filter capacitor was considered during controller tuning.

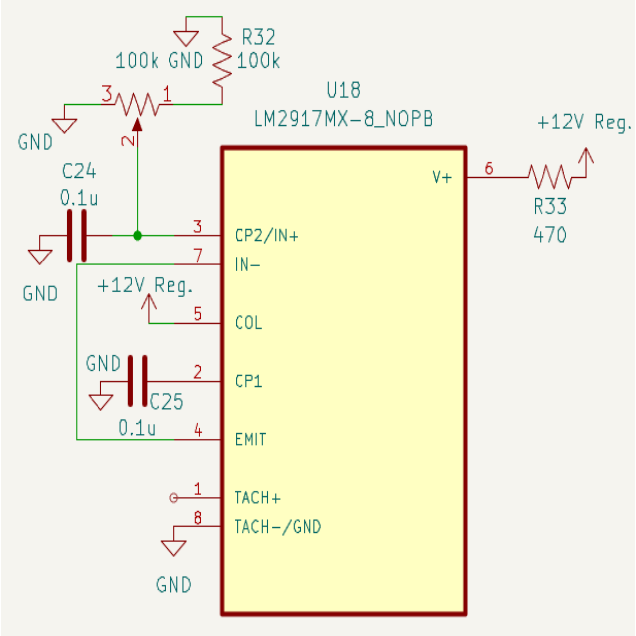

Figure 5: Implementation of F/V converter.

As seen in Figure 5, the value of capacitor C24 is 0.1μF. The equivalent resistance seen by the capacitor was $122k\Omega$, coming from the potentiometer and resistor R32 . The F/V converter pole was found with the RC low-pass filter equation:

$$\begin{aligned} f &= \frac{1}{2\pi RC} \\ &= \frac{1}{2\pi(122\,\mathrm{k\Omega})(0.1\,\mu\mathrm{F})} \\ &= 13\,\mathrm{Hz} \end{aligned} \tag{13}$$

Since this pole was very close to the dominant motor pole at 11.73 Hz, it had to be considered during tuning because it caused the feedback signal to respond more slowly in the same frequency range as the motor's dominant pole.

3.3.3 Initial Gain Selection

The dominant motor pole at 11.73 Hz and the F/V converter pole at 13 Hz defined the low frequency dynamics most relevant to the controller. Given the PWM, gate driver, and MOSFET operated in the 40 kHz range—considerably faster than the speed control loop—they could safely be neglected during gain selection.

Using the motor and feedback path dynamics, MATLAB was used to create initial estimates for the proportional and integral gains. After placing the zero at the dominant motor pole of 11.7 Hz

and estimating a crossover frequency of 10 Hz, the initial gain estimates were

$$k_p = 0.29$$

and

$$k_i = 21.61.$$

The MATLAB script also provided an estimated closed-loop step response.

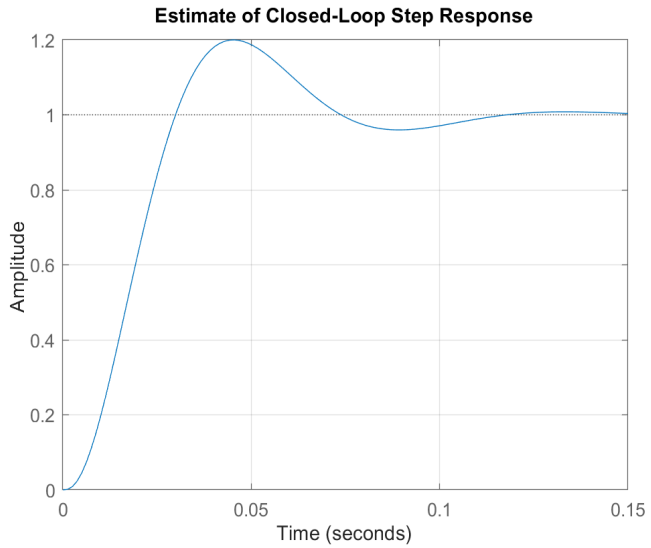

Figure 6: Estimate of closed-loop step response generated via MATLAB.

Figure 6 shows the estimated step-response would have an overshoot of approximately 20%. However, the steady-state error was minimal. Therefore, the preliminary values of C_I, $R_{in,I}$, $R_{in,P}$, and $R_{fb,P}$ were chosen to match the estimated gain values. These values are provided in Table 1.

Table 1: Preliminary component values used to implement the estimated PI controller.

Component	Symbol	Value	Units
Proportional input resistance	$R_{in,P}$	34.5	kΩ
Proportional feedback resistance	$R_{fb,P}$	10	kΩ
Integrator input resistance	$R_{in,I}$	100	kΩ
Integrator capacitor	C_I	0.47	μF

3.3.4 Manual Tuning

Testing began after setting the potentiometers of the P and I stages to their estimated values. The Analog Discovery 3 USB oscilloscope wave generator was used to inject a 500 mHz, 4 V square wave into the controller's setpoint, while the scope probes measured the setpoint against the feedback

voltage. Through observation, the P and I stage potentiometers were manually adjusted until a satisfactory step response was found. After the step response met specifications, the potentiometers were measured to find the final component values.

Table 2: Final component values after manual tuning.

Component	Symbol	Value	Units
Proportional input resistance	$R_{in,P}$	7.5	kΩ
Proportional feedback resistance	$R_{fb,P}$	10	kΩ
Integrator input resistance	$R_{in,I}$	77.3	kΩ
Integrator capacitor	C_I	0.47	μF

The component values listed in Table 2 created gain parameters:

$$k_p = 1.33$$

and

$$k_i = 27.52.$$

Using the formula

$$f_z = \frac{k_i}{2\pi k_p}, \tag{14}$$

the new zero was calculated to be 3.29 Hz. Having the zero set below the poles results in a fast, stable response with increased overshoot [4]. As the primary objective in this project is stability, this would be adequate.

Using the calculated transfer functions and the measured gains, MATLAB was used to estimate the open and closed-loop Bode plots.

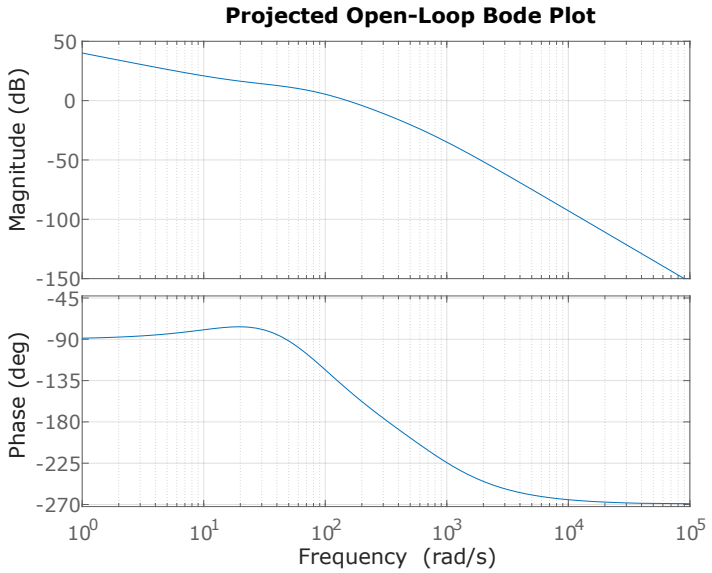

Figure 7: Estimate of open-loop Bode plot using measured parameters.

Figure 7 shows a projected crossover frequency of 24.5 Hz. Crossover frequency is the frequency where the magnitude of the open-loop transfer function crosses the 0 dB threshold. This is relevant because it provides an estimate of the effective bandwidth of the control loop. A higher crossover frequency generally corresponds to a faster response time because the controller is able to respond across a wider range of frequencies. The downside to a higher crossover frequency is that it makes the loop more sensitive to phase lag created by the poles in the system. The projected open-loop phase margin was approximately $35°$. Phase margin indicates if the system is close to instability—a larger phase margin indicates a more stable response while a smaller phase margin increases overshoot and the likelihood of oscillation. These estimations proved to be accurate. With both poles being below the crossover frequency, they both contributed significant phase lag which reduced the phase margin. This is consistent with the measured results, as the system performed well with steady-state speed regulation but had fairly aggressive overshoot, and was prone to oscillation if tuned improperly.

3.4 Triangle Wave Generation

One of the primary components of generating a PWM signal is the triangle carrier wave. The high frequency triangle wave is compared to the control voltage using a comparator, creating a square wave that is "high" if the magnitude of the control voltage is larger than the instantaneous amplitude of the triangle wave, and "low" if the magnitude is smaller. For example, in this system the control voltage could sit between 0–12 V. Using a 0–12 V triangle wave, a control voltage of 6 V would create a square wave with a duty cycle of 50%.

Initially a comparator and integrator was used to try and develop a triangle wave. However, that approach was abandoned due to issues with biasing the triangle wave. Due to its availability, the

XR2206 Monolithic Function Generator was used to create the triangle wave instead.

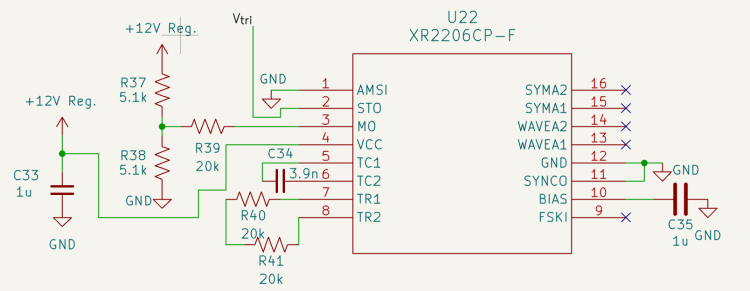

Figure 8: Realization of the XR2206 as a triangle wave generator.

The XR2206 was configured in accordance with its datasheet as seen in Figure 8 [5]. This design prioritized high frequency, 50% duty cycle and waveform stability. A high frequency was necessary in order to build a proper PWM signal, the motor in use is a brushless DC motor, so a frequency greater than 20 kHz was required. 50 kHz was chosen arbitrarily.

Frequency is set using the capacitor on Pins 5 and 6 and the resistors on Pin 7 and 8, which also set the duty cycle.

$$f = \frac{2}{C}\frac{1}{R_1 + R_2} \tag{15}$$

$$DC = \frac{R_1}{R_1 + R_2} \tag{16}$$

The datasheet also specified that the maximum output amplitude was 6 V_{pp} which would not be sufficient for the 0–12 V triangle wave required by the system. Therefore, an op-amp level shifter based on a design from Maxim Integrated was implemented to adjust the triangle wave to the required range [6].

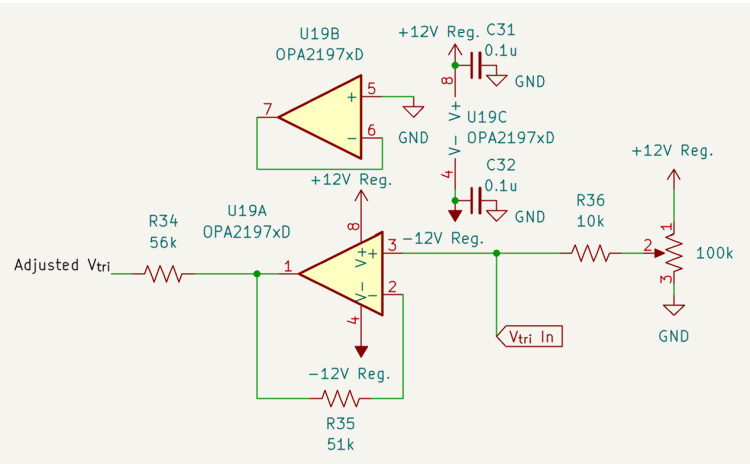

Figure 9: Realization of the op-amp level shifter.

As seen in Figure 9, an OPA2197 took the XR2206 triangle wave as an input at the non-inverting

node. On the same node, a potentiometer and fixed 10 kΩ resistor are used to set a DC bias of about -3 V—this is adjustable because we needed to tune it, shifting the triangle wave to be centered at 6 V. This resulted in a range of about 1.4 V to 10.6 V. These two values seemed to be the limits of what the IC could output, which prompted an external gain stage. In order to top off the signal, a non inverting op amp using 51 kΩ and 56 kΩ resistors were used to add a $1.1\frac{V}{V}$ gain. This brought the final range to about 1.25 V - 11.5 V, as seen in Figure 10. The gap at the bottom and top of the range was intentional, allowing for the motor to run at full speed and come to a full stop.

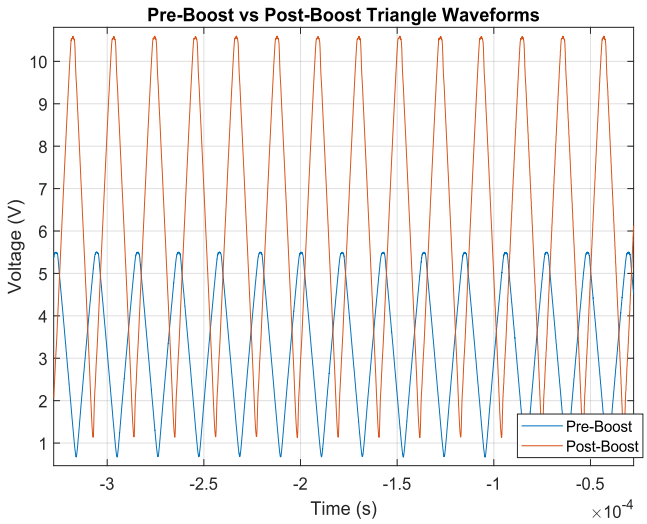

Figure 10: Triangle waveform before and after shift and boost.

3.5 Feedback Design

The feedback system consists of a Frequency to Voltage Converter and op amp. The chip chosen was a LM2917. Due to its built in filter, the gain is predictable—set using a resistor and capacitor—and it has stability in the output voltage because of the built in Zener diode that keeps the supply voltage separate from the output. These features were specifically important to this project's design, as minimizing the effects of ripple from the encoder frequency and supply noise would be imperative for a clean DC feedback output. Additionally, the implementation of this IC required only a small number of external components, simplifying both the design and the soldering associated.

The design process is as follows. A frequency from the motor, acquired from the built-in encoder, needs to be scaled to a 0-12 V DC signal to be used in the error calculation. However, due to limitations of the LM2917 chip, an extra gain step was necessary. The output of the LM2917 can only vary from 0 to 6.6 V. This necessitated a gain of

$$\frac{12}{6.6} = 1.8\frac{V}{V}$$

via an extra non-inverting op amp stage to scale it properly for usage in the error calculation.

A linear scaling is required for this implementation, meaning that 0-3 kHz should match a 0-6.6 V scale. This resulted in a conversion rate of $\frac{3000}{6.6} => 450\frac{Hz}{V}$.

The frequency to voltage ratio in the converter is set using the following equation.

$$V_{out} = V_{ref} R C_2 f_{max}, \tag{17}$$

The LM2917 has a V_{ref} of 7.56 V due to the internal Zener diode. This is briefly mentioned in the datasheet [7]. Frequency f_{max} is the maximum encoder output, 3 kHz. This is a measured value. In order to set V_{out} to 12 V, these values can be substituted and then R and C_1 can be chosen correspondingly.

$$\begin{aligned} 12V &= (7.56V)(RC_1)(3000Hz) \\ 0.0005291 &= RC_1 \end{aligned} \tag{18}$$

Many considerations had to be made in the process of selecting R and C_1. The first is that resistor sizing affects response linearity. Testing revealed that any values lower than 66 kΩ resulted in unreliable scaling. Due to many unsuccessful assembly attempts, the final process was to follow the provided schematic as much as possible, which meant selecting an R of 100 kΩ and C of 0.02μF This resulted in a scaling close to what was necessary, but not exactly correct. This is why the potentiometer was added in series with the 100 k resistor, to allow for fine tuning because the scaling isn't perfectly linear and changed quickly with different values. The tuning was done by inputting a few frequencies—1 kHz, 2 kHz, 2.5 kHz and 3 kHz—into the converter and testing the output voltage. Once completed, the R value was about 122 kΩ. It can be seen that these values do not properly match the derivation above. This is assumed to be due to the difference in V_{ref} that is seen, listed at 7.56 V and measured at 6.6 V.

This design results in a linear scaling from 0-3 kHz to 0-6.6 V. Following this, a simple non inverting op amp constructed using 100 kΩ and 82 kΩ resistors are used to set a gain of 1.8 to scale the output from a maximum of 6.6 V to 12 V.

4 Results

4.1 Methods and Testing Techniques

The primary methods of specification verification were step-response tests and manual loading tests. Step-response tests were performed by injecting periodic square waves of varying amplitudes with the Digilent waveform generator and measuring the response with the Digilent oscilloscope. For the load testing, the controller was brought to steady-state, then the motor shaft was manually loaded by grabbing the attached aluminum wheel while the setpoint and feedback voltages were compared using the oscilloscope. The current supply specification of at least 1 Amp was verified by powering the system with a bench-top power supply, applying load to the motor, and visually confirming the current through the system exceeded 1 Amp. Additionally, the motor-driving components chosen had current ratings far exceeding 1 Amp.

4.2 No-Load Step Response

To verify the specification of motor speed being within $\pm 10\%$ of setpoint at steady-state from 50 to 150 RPM, square waves of varying amplitudes within the operating range were injected into the controller.

Table 3: Approximate Voltage Values and Corresponding Speed Setpoints

Approximate Voltage (V)	**Approximate Speed Setpoint (RPM)**
2	50
4	100
6	150

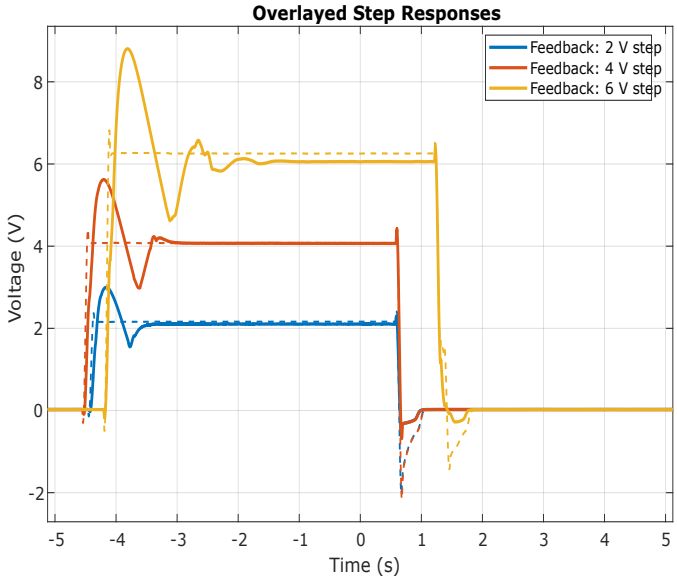

Figure 11: Step responses of the controller with a 2 V, 4 V, and 6 V setpoints.

Through the 2 V, 4 V, and 6 V step-response tests shown in the figure above, steady-state error stayed below 4%, while overshoot ranged between 30–33% and settling time ranged between 1.4–1.7 seconds. For testing the steady-state speed under load, the controller was allowed to reach steady-state in the same range of setpoints. A load was then manually applied to the motor shaft. The oscilloscope was used to capture a 10 second window to demonstrate the initial load followed by recovery.

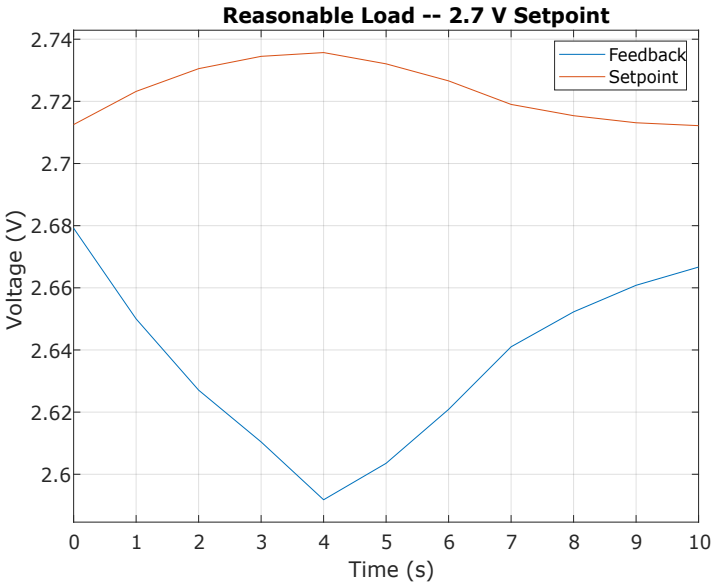

Figure 12: Response of the controller under reasonable load with 2.7 V setpoint.

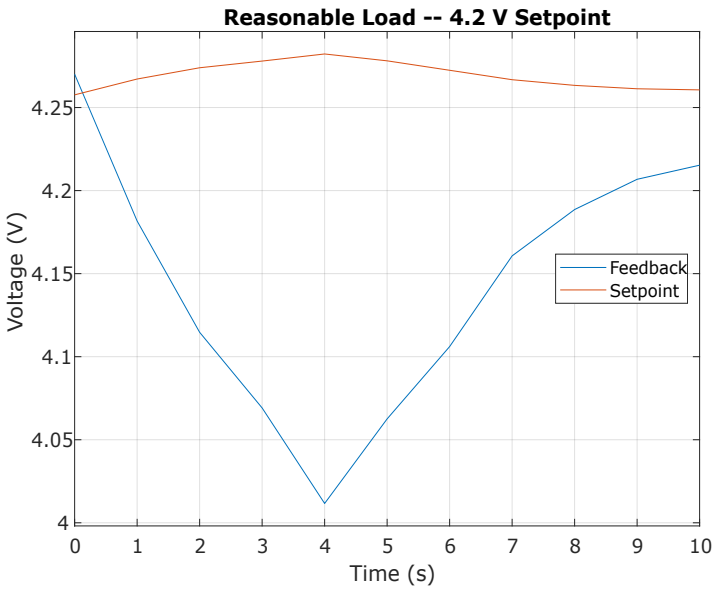

Figure 13: Response of the controller under reasonable load with 4.2 V setpoint.

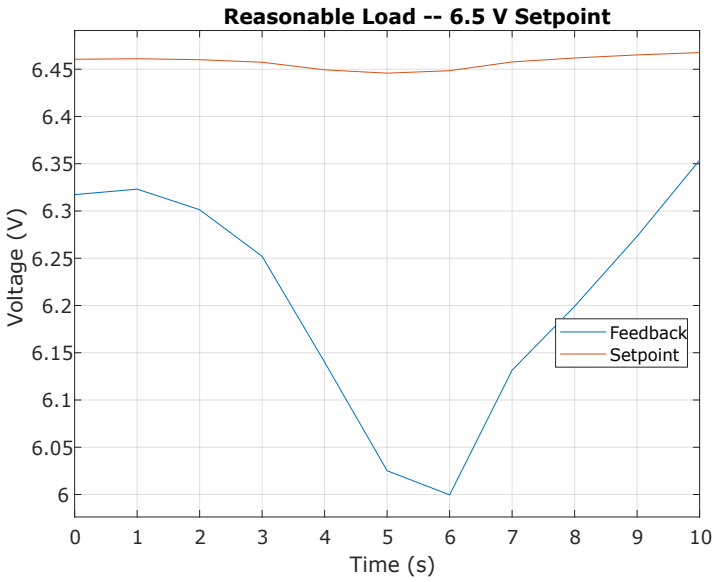

Figure 14: Response of the controller under reasonable load with 6.5 V setpoint.

The figures above show the motor running at steady-state, experiencing a reasonable load, then recovering to steady-state. As demonstrated, the controller was able to recover to steady-state, keeping error below 4% in each case. The experimental results are summarized in the table below.

Table 4: Summary of experimental results.

Test Condition	Setpoint (V)	Steady-State Error (%)	Overshoot (%)	Settling Time (s)
Step response	2.0	2.82	33	1.4
Step response	4.0	0.34	31.6	1.4
Step response	6.0	3.73	33.7	1.7
Loaded response	2.7	3.77	—	—
Loaded response	4.2	1.06	—	—
Loaded response	6.5	1.72	—	—
Average steady-state error		2.24	—	

4.2.1 Discussion

These results satisfy the objectives of the project by keeping steady-state speed within $\pm 10\%$ of setpoint both in general and under reasonable load. The team is confident in the measuring methods and data analysis, especially because the behavior of the controller projected via MATLAB is similar to the observed behavior. This is especially true with overshoot—the average overshoot in the step response was 32.7% compared to the projected 33%.

There were several issues that arose during the pursuit of satisfactory results. Initially a lot of issues arose with designing the triangle wave generator and frequency to voltage converter. The triangle wave was initially designed using a comparator and an integrator, however this was abandoned after it was found to be very difficult to bias the circuit. The frequency to voltage converter posed multiple issues. The datasheet did not provide clear information about the LM2917 model and that certain parts of the chip changed its operation, like the Zener diode changing the reference voltage of the chip. Initially designs would work and then after a few days of operation would drift away from linearity or become completely inaccurate, and this was due to negligence of two equations hidden deep within the datasheet.

Towards design review it became more apparent that the team lacked an understanding of how the system's poles interacted with each other. One example is the LM2917 pole that was disregarded. At design review the PI controller was half operational, the P stage could be changed but would not have a significant effect on the response, however adjusting the integrator input resistance potentiometer above 10Ω resulted in uncontrolled oscillation and instability. This was due to the misplaced poles across the circuit and lack of understanding of the PI controller pole and zero and its interactions with the motor mechanical and electrical poles. Once resolved, the operation of the system was much more robust and stable.

Additionally, a lot of time was wasted attempting to accurately model the system in Micro-Cap. The PI controller could be modeled, but modeling the DC motor and feedback proved to be exceptionally challenging and the simulations were eventually abandoned. This lead to a less informed and more uncertain build stage.

Given the opportunity to start again, the team would make three changes to the final project.

Primarily, finding a way to raise the F/V converter pole without compromising operation. This would have reduced lag in the feedback loop, allowing for a less aggressive tuning of the controller. Second would be building the final project on a PCB. A lot of time could have been saved for proper troubleshooting if the circuit did not need to be hand-soldered on breadboards. Finally, the feedback system would have utilized current sensing rather than the encoder. Converting the encoder frequency to a usable signal proved to be more difficult than expected.

5 Conclusion

This project demonstrates the complete development of a purely analog closed-loop speed control system designed to drive a DC motor under load. The system utilized a PI controller, analog PWM generation, a gate driver and MOSFET stage, and an encoder based feedback loop. The controller was capable of regulating motor speed without the use of a microcontroller. The final product satisfied the design requirements established for the project. Bench testing proved that the controller could maintain steady-state speed within $\pm 10\%$ of setpoint over the operating range of 50 to 150 RPM, both under no-load conditions and with reasonable load applied. Additionally, the motor driver was able to meet the power specification of supplying at least 1 Amp to the motor. The project also provided the team with insight into analog design and control theory.

References

[1] Texas Instruments, *OPAx197 36-V Precision Rail-to-Rail Input/Output Operational Amplifiers Datasheet*, Texas Instruments, March 2018. [Online]. Available: https://www.ti.com/lit/ds/symlink/opa2197.pdf

[2] A. Pini, "Analog integrators: How to apply them for sensor interfaces, signal generation, and filtering," DigiKey, August 2020. [Online]. Available: https://www.digikey.com/en/articles/analog-integrators-how-to-apply-them-for-sensor-interfaces

[3] Dc motor speed: System modeling. Control Tutorials for MATLAB and Simulink, University of Michigan. [Online]. Available: https://ctms.engin.umich.edu/CTMS/?example=MotorSpeed&section=SystemModeling

[4] K. J. Astrom and R. M. Murray, "Pid control," Chapter draft in *Feedback Systems*, August 2019, [Online]. Available: https://www.cds.caltech.edu/~murray/books/AM08/pdf/fbs-pid_17Aug2019.pdf [Accessed: Mar. 12, 2026].

[5] EXAR, *XR2206 Monolithic Function Generator*, EXAR, February 2008. [Online]. Available: https://cdn.sparkfun.com/assets/8/a/b/3/9/XR2206.pdf

[6] Maxim Integrated, "Single-supply op amp forms noninverting level shifter," Maxim Integrated, February 2011. [Online]. Available: https://www.analog.com/media/en/technical-documentation/design-notes/singlesupply-op-amp-forms-noninverting-level-shifter.pdf

[7] Texas Instruments, *LM2907 and LM2917 Frequency to Voltage Converter*, Texas Instruments, December 2016. [Online]. Available: https://www.ti.com/lit/ds/symlink/lm2907-n.pdf

A PI Controller Transfer Function Derivations

A.1 Proportional Stage Derivation

The proportional stage was implemented as an inverting amplifier. Assuming an ideal op-amp, the non-inverting input is grounded and the inverting input is at virtual ground. The input resistance of the proportional stage was implemented as a fixed resistor in series with a potentiometer wired as a variable resistor. Therefore,

$$R_{in,P} = R_{fixed,P} + R_{pot,P}. \tag{19}$$

Applying Kirchhoff's Current Law at the inverting input gives,

$$\frac{V_e(s)}{R_{in,P}} + \frac{V_P(s)}{R_{fb}, P} = 0 \tag{20}$$

where $V_P(s)$ is the output voltage of the proportional stage.

Rearranging yields:

$$\frac{V_e(s)}{R_{in,P}} = -\frac{V_{o,P}(s)}{R_{fb,P}}. \tag{21}$$

Solving for the transfer function of the proportional stage gives:

$$H_P(s) = \frac{V_P(s)}{V_e(s)} = -\frac{R_{fb,P}}{R_{in,P}} \tag{22}$$

Substituting the input resistance gives

$$H_P(s) = -\frac{R_{fb,P}}{R_{fixed,P} + R_{pot,P}}. \tag{23}$$

A.2 Integral Stage Derivation

The integral stage was implemented as an inverting integrator. Assuming an ideal op-amp, the non-inverting input is grounded and the inverting input is at virtual ground. The input resistance of the proportional stage was implemented as a fixed resistor in series with a potentiometer wired as a variable resistor. Therefore,

$$R_{in,I} = R_{fixed,I} + R_{pot,I}. \tag{24}$$

The feedback network consisted of a capacitor C_I in parallel with a bleed resistor R_{bleed}. The impedance of the capacitor is

$$Z_{C_I}(s) = \frac{1}{sC_I} \tag{25}$$

Since the capacitor is in parallel with the bleed resistor, the feedback network impedance is expressed as:

$$Z_{fb,I}(s) = Z_{C_I}(s) \parallel R_{bleed}. \tag{26}$$

Substituting the capacitor impedance gives:

$$Z_{fb,I}(s) = \frac{(\frac{1}{sC_I})R_{bleed}}{(\frac{1}{sC_I}) + R_{bleed}}. \tag{27}$$

After simplifying,

$$Z_{fb,I}(s) = \frac{R_{bleed,I}}{1 + sC_I R_{bleed}}. \tag{28}$$

Letting $V_{o,I}$ be the output voltage of the integral stage, the transfer function is:

$$H_I(s) = \frac{V_{o,I}(s)}{V_e(s)} = -\frac{Z_{fb,I}(s)}{R_{in,I}(s)}. \tag{29}$$

Substituting the feedback impedance and the input resistance yields:

$$H_I(s) = -\frac{R_{bleed}}{(R_{fixed,I} + R_{pot,I})(1 + sC_I R_{bleed})} \tag{30}$$

B Derivation of Motor Transfer Function and Poles

B.1 Transfer Function

The motor parameters used in the transfer function development were taken from the datasheet, manufacturer information, and calculated.

Table 5: DC motor parameters used for transfer-function development.

Parameter	Symbol	Value	Units	Source
Armature inductance	L	2.3	mH	Provided
Armature resistance	R	2.2	Ω	Provided
Back EMF constant	K_e	0.258	V·s/rad	Calculated
Torque constant	K_t	0.258	N·m/A	Provided
Viscous friction coefficient	B	1.53×10^{-3}	N·m·s/rad	Calculated
Rotor inertia (approx.)	J	4.7×10^{-4}	kg·m^2	Calculated

The back EMF at the maximum efficiency of the motor was estimated as

$$\begin{aligned} V_{\text{emf}} &= V_s - IR_a \\ &= 12 - (0.78)(2.2) \\ &= 10.284 \text{ V} \end{aligned} \tag{31}$$

The torque constant was found as the inverse of the inverse of the slope of the current versus torque curve provided by the manufacturer:

$$I(\tau) = 0.11 + 0.038\tau \tag{32}$$

which gives a slope of

$$\frac{dI}{d\tau} = 0.038 \text{ A}/(\text{kg} \cdot \text{mm}). \tag{33}$$

Given the information provided, the torque constant was estimated as

$$\begin{aligned} K_t &= \left(\frac{dI}{d\tau}\right)^{-1} \\ &= \frac{1}{0.038} \\ &= 26.32 \text{ kg} \cdot \text{mm/A}. \end{aligned} \tag{34}$$

Converting to SI units,

$$\begin{aligned} K_t &= 26.32\left(9.81 \cdot 10^{-3}\right) \\ &= 0.258 \text{ N} \cdot \text{m/A}. \end{aligned} \tag{35}$$

For the final motor model, the assumption that $K_e = K_t$ was made. Using the standard model of a DC motor, the electrical equation was

$$V(s) = (Ls + R)I(s) + K_e\omega(s) \tag{36}$$

and the mechanical equation was

$$K_t I(s) = (Js + B)\omega(s). \tag{37}$$

Solving the mechanical equation for current gives

$$I(s) = \frac{Js + B}{K_t}\omega(s). \tag{38}$$

Substituting into the electrical equation gives

$$\begin{aligned} V(s) &= (Ls + R)\left(\frac{Js + B}{K_t}\right)\omega(s) + K_e\omega(s) \\ &= \omega(s)\left[\frac{(Ls + R)(Js + B)}{K_t} + K_e\right]. \end{aligned} \tag{39}$$

Therefore, the total motor transfer function is

$$G_m(s) = \frac{\omega(s)}{V(s)} = \frac{K_t}{(Ls + R)(Js + B) + K_t K_e}. \tag{40}$$

Expanding the denominator and substituting the parameter values gives

$$\begin{aligned} JL &= (4.7 \cdot 10^{-4})(2.3 \cdot 10^{-3}) \\ &= 1.22 \cdot 10^{-6}, \end{aligned} \tag{41}$$

$$\begin{aligned} JR + BL &= (4.7 \cdot 10^{-4})(2.2) + (1.53 \cdot 10^{-3})(2.3 \cdot 10^{-3}) \\ &= 1.04 \cdot 10^{-3}, \end{aligned} \tag{42}$$

$$\begin{aligned} BR + K_t K_e &= (3.37 \cdot 10^{-3}) + (0.257)^2 \\ &= 0.07. \end{aligned} \tag{43}$$

Therefore, the final motor transfer function used for the controller design was

$$G_m(s) = \frac{0.257}{(1.22 \cdot 10^{-6})s^2 + (1.04 \cdot 10^{-3})s + 0.07}. \tag{44}$$

B.1.1 Finding Poles

Given the transfer function, the poles could be found from the denominator:

$$(1.22 \cdot 10^{-6} s^2) + (1.04 \cdot 10^{-3})s + 0.07 = 0. \tag{45}$$

Using the quadratic formula:

$$\begin{aligned} s &= \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \\ &= \frac{(-1.04 \cdot 10^{-3}) \pm \sqrt{(1.04 \cdot 10^{-3}))^2 - 4(1.22 \cdot 10^{-6})(0.07)}}{2(1.22 \cdot 10^{-6})} \end{aligned} \tag{46}$$

Solving gives the poles:

$$s = -73.68\text{rads/sec}, -778.8\text{rads/sec}. \tag{47}$$

Converting to frequency gives

$$\begin{aligned} f &= \frac{|s|}{2\pi}, \\ f &= \frac{73.68}{2\pi} = 11.73\text{Hz}, \\ f &= \frac{778.8}{2\pi} = 123.95\text{Hz}, \end{aligned} \tag{48}$$

with the slower pole of 11.73Hz being the dominant pole.